

The Vault Keeper: A Private, Self-Hosted Alternative to Public Cloud Storage

By: Taha Anwar, Tomas Chaparro, John-Paul Osele, and Cole Roberts

Table of Contents

Project Proposal	2
Project Abstract	3
High Level Requirement	3
Conceptual Design	3
Proof of Concept	4
Background	4
Required Resources	4
Project Design	4
Vision	5
Persona #1 – Elliot Alderson, A Junior Software Developer	5
Persona #2 – Sarah Williams, A Graduate in Accounting & Finance	5
Persona #3 – Max Caulfield, A High School Senior & Multimedia Art Student	6
Persona #4 – Norman Jayden, An Independent Legal Consultant & Compliance Auditor	7
Persona #5 – Lee Everett, A Researcher in Anthropology	8
Class Diagram	9
Sequence Diagram	10
Automated Test Results	11
Project Progress	12
Week 1 Progress	12
Week 2 Progress	14
Week 3 Progress	16
Week 4 Progress	18

Project Proposal

Project Abstract

The Vault Keeper is a self-hosted cloud storage platform designed to provide data independence by eliminating 3rd party data mining and providing free customization. While products like Google Drive and Dropbox already exist, users don't have full freedom to customize and use these services. With "The Vault Keeper," I propose a lightweight and open-source solution that allows users to customize and manage their own data for free. The software's main use is to provide an environment that allows users to upload, store, and manage their files on a private Linux server. The file management will be seamless and provide log file data to track changes to files. This software is intended for people who care more deeply about their digital privacy, developers looking for secure and internal file sharing, and users looking for completely free and customizable alternatives to existing products. This project demonstrates good security practices and offers users a secure environment for internal data handling.

High Level Requirement

The Vault Keeper allows users to securely access a private storage environment to upload, manage, and download personal files through an intuitive web interface. The system ensures data privacy by isolating storage on a local docker container.

Conceptual Design

The Vault Keeper is built on a high-performance, client-server architecture designed for privacy and environmental isolation. The system is split into a zero-knowledge frontend and a high-concurrency backend to ensure that sensitive data is encrypted before it ever leaves the user's local environment.

The system uses a model where the storage backend resides on a private local docker container, isolated from the user's primary hardware to protect against local failures or malware.

- Frontend: All encryption happens client-side. The server acts as a vault that stores encrypted files without ever possessing the keys to read them.
- Backend: The backend manages the intake of these encrypted files by organizing the metadata in the database and handles the activity log auditing.

With the decision to prioritize performance and modern web standards, the following tools have been selected:

Category	Tool	Reasoning
Frontend	React (Vite) + Tailwind	For fast development, easy Dark Mode implementation, and PWA support.
Backend	FastAPI (Python)	Selected over Flask for its high performance with file streams and greater asynchronous support.
Database	SQLite	A simple, portable, and file-based database that requires no extra server overhead.
Storage	Local Linux Directory	The most direct way to handle large file volumes on a self-hosted Ubuntu server.

Category	Tool	Reasoning
Auth/Security	Web Crypto API	Implements client-side encryption to ensure the server remains zero-knowledge.

Proof of Concept

Link to repository: <https://github.com/cis3296s26/The-Vault-Keeper>

Link to Live Beta: <https://the-vault-keeper.vercel.app/>

This proof of concept was done on a Windows environment to check framework compatibility. Currently, a live beta is deployed using Vercel to demonstrate React/FastAPI integration. In the final project, the Ubuntu Server will serve as a separate, standalone deployment environment. With this key element, the application logic and data storage are isolated from the user's machine, providing environmental isolation. A README.md is added with System Specifications & Compatibility as well as instructions on how to run.

Background

The Vault Keeper addresses concerns with cloud platforms and data ownership with similar platforms like Google Drive lacking transparency. While there are existing alternatives, they are often too complex for a single user's needs. The Vault Keeper is a project designed to be a simpler, lightweight version of these existing services. By building this from scratch with free tools, I am proposing a Client-Server architecture where the local docker container acts as a separate deployment environment from the user's local machine. Using FastAPI to manage this architecture allows for efficient data isolation, protecting files from hardware failures or malware. This provides users with a personal and transparent way to manage their data across a network rather than storing it on a local hard drive.

Required Resources

- Software & Frameworks: Python 3.10+ with FastAPI for the backend and React (Vite) for the frontend.
- Infrastructure: Ubuntu Server 22.04 LTS hosted via VirtualBox for environmental isolation.
- Data & Security: SQLite for metadata/logs and Web Crypto API for client-side encryption.
- Hardware & Tools: GitHub for version control and a PC/Laptop with at least 8GB RAM to handle the development tools and VM simultaneously.

Project Design

Vision

For everyday people who care about what happens with their personal data and want to take control of how that data is managed. The ‘Vault Keeper’ provides a customizable application that allows users to track and manage personal files on a free and private Linux server. Unlike traditional file managers that store these files for the user, ‘The Vault Keeper’ is solely managed by the user. Our product uses this advantage to provide security to the user, as they know that they are in control and third parties won’t have access to their data.

Persona #1 – Elliot Alderson, A Junior Software Developer

Name: Elliot Alderson

Age: 26

Occupation: Junior Software Developer at a mid-sized fintech company

Educational background:

- Bachelor’s degree in Computer Science
- Completed several online certifications in cybersecurity and Linux systems administration

Technology experience:

- Highly proficient
- Their daily driver is Linux (Arch)
- Comfortable with the command line, Git, and self-hosting services on a home server
- Actively avoids cloud services that monetize user data

Demographics:

- Technically motivated hobbyist who runs a home lab
- Part of several open-source and privacy-focused online communities (e.g., r/selfhosted, r/privacy)
- No family dependents, so the storage is purely personal and professional

Why would they use the vault keeper:

- Store personal projects, credentials, and sensitive documents without trusting a third party like Google or Dropbox
- Wants to see exactly what the software does; appreciates that it’s open-source and lightweight
- Values the activity log feature to audit exactly who accessed or modified what files
- Would likely contribute feedback or even pull requests to the project

Potential difficulties:

- May want more advanced features (e.g., AI integration to search files just from a natural language description) that aren’t available in early versions
- He could also become frustrated if the documentation is sloppy or imprecise

Persona #2 – Sarah Williams, A Graduate in Accounting & Finance

Name: Sarah Williams

Age: 23

Occupation: None; working on finding a job in accounting

Educational background:

- Bachelor's degree in Accounting & Finance

Technology experience:

- Very familiar with excel and has dabbled in the language R in one of her Economics classes
- Has no difficulties using her phone or laptop, but is slow to learn new software
- Likes it when the apps she downloads are easy to use and provide user guides

Demographics:

- Just graduated and has gone back home to live with her parents until she finds a job
- Looking to live and work in a big city on the East Coast (she's been on the West Coast her whole life)
- Lives on a tight budget, even at home with her parents, because she knows her student loans are going to eat a lot of her salary

Why would they use the vault keeper:

- Sarah likes to stay organized and wants a way to store files that she can customize
- She knows that she'll be dealing with internal company files in the future, so she wants to practice keeping personal files safe at home
- She is currently on a budget and would like the fact that Vault Keeper is free to use
- She has seen many articles online about how companies steal data. She doesn't understand how it happens and would like a way to avoid it

Potential difficulties:

- Sarah is not very technical, so she might have trouble setting up the app
- If the app does not have a tutorial or guide, she won't touch it
- If the app is not easy to use and intuitive, she'll move to a different app

Persona #3 – Max Caulfield, A High School Senior & Multimedia Art Student

Name: Max Caulfield

Age: 18

Occupation: High School Senior and Multimedia Art Student

Educational background:

- Completing a high school diploma with a focus on Fine Arts and Creative Writing

Technology experience:

- Moderately experienced with apps, but not experienced with more technical uses of technology
- An expert in Procreate, Canva, and digital journaling apps
- Uses her phone and tablet a lot to capture important ideas, sketches, and diary thoughts

Demographics:

- Lives with their family with a shared family computer
- Sells customs stickers and makes art prints for the school newspaper
- Values her digital space and treats it as an extension of her personal expression, like a nichely decorated bedroom

Why would they use the vault keeper:

- Max likes to switch to Dark Mode sometimes and the ability to organize her files freely makes the Vault feel like her own curated gallery
- She uses the drive to store scanned diary pages and art that she wants to keep private
- She likes the idea of password protected files to lock away her ideas, making her feel secure that her ideas and information are written down safely, available to open at any time
- She prefers self-hosted tools because it feels more unique compared to generic public cloud services, bringing out her artistic vibe

Potential difficulties:

- She might find the idea of a Linux server setup intimidating since it's not a standard one-click install
- If there's an error in the Python backend, she won't be able to fix it without extremely simple, non-technical instructions

Persona #4 – Norman Jayden, An Independent Legal Consultant & Compliance Auditor

Name: Norman Jayden

Age: 43

Occupation: Independent Legal Consultant and Compliance Auditor

Educational background:

- Political Science BA & Juris Doctor (JD)

Technology experience:

- Moderate
- Proficient in legal research databases and digital documentation
- Uses technology as a tool for organization but is skeptical of Big Tech data-sharing policies and cloud-based Terms of Service that might compromise attorney-client information

Demographics:

- Works solo out of a home office
- Handles sensitive corporate governance documents and private litigation files for small firms
- Prefers personal control over his library and records rather than relying on subscription-based services.

Why would they use the vault keeper:

- To ensure third-party data miners cannot scan private legal contracts
- Likes the use of activity logs to prove when documents were changed
- Likes the extra layer of protection that locally hosted services provide
- Since he is very meticulous, he enjoys keeping files in a place he physically controls/customizes

Potential difficulties:

- Initially he might find managing a Linux server distracting from his legal work
- Worries that a mistake on a self-hosted platform could lead to data loss or security issues

Persona #5 – Lee Everett, A Researcher in Anthropology

Name: Lee Everett

Age: 37

Occupation: Researcher in Anthropology

Educational background:

- BA in Anthropology & Ph.D. in Anthropology

Technology experience:

- Moderate
- Expert in qualitative data software and handling notes digitally
- Often works in regions with unreliable internet, so he distrusts public clouds for storing sensitive interviews that could put people at risk

Demographics:

- Travels frequently for research
- Values safe tools that aren't hidden behind a paywall for full access

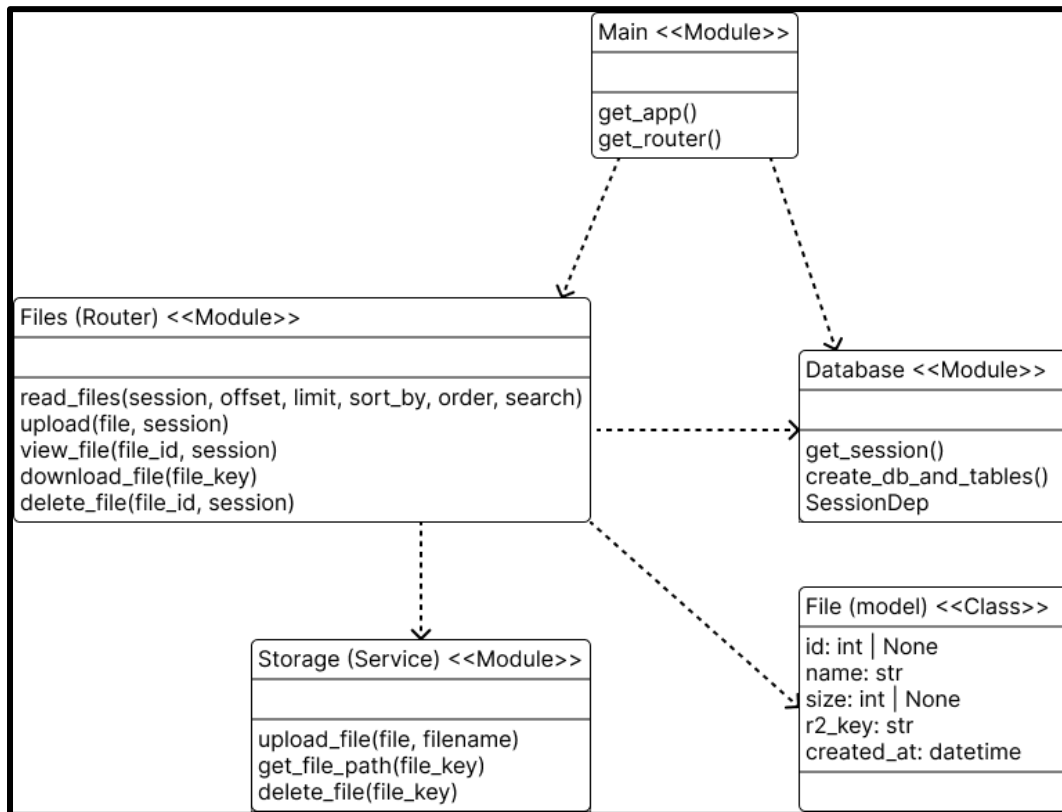
Why would they use the vault keeper:

- Needs to apply ethical research practices and anonymity of participant data
- Finds the platform useful for publishing unedited findings before public publishing
- Prefers a simple and customizable layout

Potential difficulties:

- Might have trouble accessing the tools needed for the platform while in the field, so notes will need to be passed onto the server afterward
- Needs assistance/tutorials initially, since he has been using the same software for years

Class Diagram

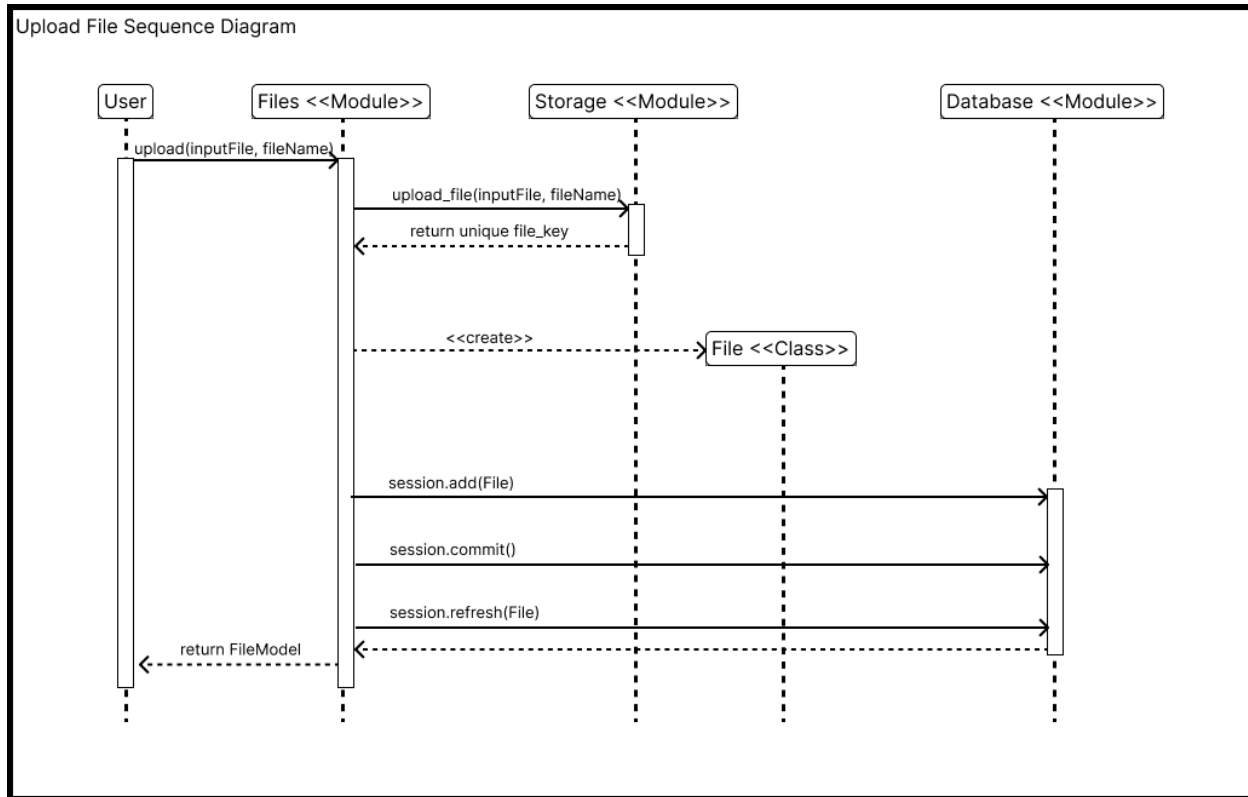


Explanation

This class diagram shows the five main parts of the app and how they connect.

- 1) Main: The entry point of the app, where everything is started by creating the app and setting up the routes.
- 2) Files (router): Handles all of the file-related actions that a user can do such as uploading, reading, viewing, downloading, and deleting files.
- 3) Storage: The model which physically deals with the files on the server. The files are uploaded, found, and deleted from the disk.
- 4) Database: This manages the connection and sessions for data that can be retrieved and saved.
- 5) File (model): The class which is the blueprint for what a file should look like inside the database. It has an ID, name, storage key, size, and when it was created.

Sequence Diagram



Explanation

This sequence diagram shows the main sequence of events between the different parts of the application.

1. The user first calls `upload()` with their file and the filename.
2. Files (router) passes it to Storage where it physically saves the file and returns a unique `file_key`.
3. A new file object is created in memory by using that key and the file's info.
4. That file gets added to the database session.
5. The session is committed; the data is permanently saved.
6. The session refreshes the file, so it has its final database ID.
7. The completed file record is then returned back to the user.

Automated Test Results

Report generated on 29-Apr-2026 at 14:09:14 by [pytest-html v4.2.0](#)

Environment

Python	3.11.15
Platform	macOS-26.3-arm64-arm-64bit
Packages	- pytest: 9.0.3 - pluggy: 1.6.0
Plugins	- cov: 7.1.0 - metadata: 3.1.1 - html: 4.2.0 - anyio: 4.13.0
JAVA_HOME	/Library/Java/JavaVirtualMachines/temurin-17.jdk/Contents/Home

Summary

17 tests took 183 ms.

(Un)check the boxes to filter the results.

☒ 0 Failed,
 ☒ 17 Passed,
 ☒ 0 Skipped,
 ☒ 0 Expected failures,
 ☒ 0 Unexpected passes,
 ☒ 0 Errors,
 ☒ 0 Reruns,
 ☒ 0 Retried,
 [Show all details](#) / [Hide all details](#)

Result 	Test	Duration	Links
Passed	tests/test_file_router.py::test_read_files_returns_no_files_on_fresh_start	19 ms	
Passed	tests/test_file_router.py::test_read_files_search_filter	17 ms	
Passed	tests/test_file_router.py::test_read_files_sort_ascending	21 ms	
Passed	tests/test_file_router.py::test_upload_returns_file_model	9 ms	
Passed	tests/test_file_router.py::test_upload_persists_in_db	10 ms	
Passed	tests/test_file_router.py::test_view_file_returns_url	11 ms	
Passed	tests/test_file_router.py::test_view_file_not_found	9 ms	
Passed	tests/test_file_router.py::test_download_existing_file	12 ms	
Passed	tests/test_file_router.py::test_download_missing_file_returns_404	7 ms	
Passed	tests/test_file_router.py::test_delete_file_returns_200	11 ms	
Passed	tests/test_file_router.py::test_delete_file_removes_from_list	14 ms	
Passed	tests/test_file_router.py::test_delete_nonexistent_file_returns_404	8 ms	
Passed	tests/test_storage.py::test_upload_file_returns_key	3 ms	
Passed	tests/test_storage.py::test_upload_file_creates_file_on_disk	1 ms	
Passed	tests/test_storage.py::test_get_file_path_returns_correct_path	1 ms	
Passed	tests/test_storage.py::test_delete_file_removes_from_disk	1 ms	
Passed	tests/test_storage.py::test_delete_file_nonexistent_does_not_error	0 ms	

These passed tests show that all 17 automated tests ran successfully. These tests confirm that the app can handle file operations such as uploading, downloading, viewing, and deleting files. Edge cases are also handled like downloading or deleting a file that doesn't exist.

Project Progress

Week 1 Progress

Scrum Master: Taha Anwar & John-Paul Osele

Sprint Goal: Finalize the core system architecture, start creating the database backend, and start the initial frontend prototype for beta testing.

Backlog Features

- Web Interface: Initial React frontend and Figma prototype for file management
- Database & Storage: SQLite setup and FastAPI backend for handling file uploads
- Security Tools: Client-side encryption research and a random password generator
- System Documentation: Finalizing the Project Proposal and UML design diagrams
- Project Documentation: Renewed proposal and high-fidelity UML diagrams.
- Security & Authentication: Random password generation and initial authentication logic.
- Testing: Comprehensive beta testing phase for the storage environment.

Tasks in Sprint	Task Status at end of Sprint	Assigned To
#34 – Set up database for the backend	Completed	Cole Roberts
#23 – Start making front end	Completed	Cole Roberts
#33 – Create Figma prototype and start on frontend	Completed	Cole Roberts
#25 – Make progress on the backend	Completed	Taha Anwar
#21 – Fix the release issue	Completed	Cole Roberts
#24 – Research optimal tools for UI/backend	Completed	Taha Anwar / John-Paul Osele
#14 – Random Password Generator	Completed	Tomas Chaparro

#32 – Final Tech Stack	In-Progress	Taha Anwar / John-Paul Osele
#16 – Password Protected Files	In-Progress	Tomas Chaparro
#52 – Create Documentation File	Completed	Taha Anwar
#49 – Renew Project Proposal	Completed	Taha Anwar
#50 – Add first UMLs for class / sequence	Completed	Taha Anwar
#51 – Complete Beta Testing + Documentation	Completed	Taha Anwar
#14 – Random Password Generator	Completed	Tomas Chaparro
#34 – Set up database for the backend	Completed	Cole Roberts
#33 – Create Figma prototype/frontend	Completed	Cole Roberts
#15 – File Filtering and Sorting	In-Progress	Tomas Chaparro
#31 – Add authentication	Not Started	Team

Estimated Velocity (At beginning of Sprint)	27
Calculated Velocity (At end of Sprint)	19

Week 2 Progress

Scrum Master: Cole Roberts

Sprint Goal: Improve upon the backend and implement the frontend based on the existing backend. Also, documenting the architecture and setting up for the implementation of the final features list.

Backlog Features

- UI/UX Finalization: Completing the UI/UX (app experience) visually for the frontend
- Linux Research: Researching server-side storage of files, how it works, and how to integrate Linux
- Advanced File Handling: The implementation of multiple-file uploads and password protected files for selected files
- System Feedback: Adding confirmation modals for user interaction and intuitiveness

Tasks in Sprint	Task Status at end of Sprint	Assigned To	Size
#45 – Add confirmation modals (V2)	In Progress	Cole Roberts	3
#54 – Finalize UI/UX Design	Complete	Cole Roberts	8
#42 – Make mobile friendly	In Progress	Cole Roberts	5
#42 – Change website title and favicon	In Progress	Cole Roberts	1
#55 – Complete Week 2 Scrum Review Documentation	Complete	Taha Anwar	3
#30 – Password Protected Files	In Progress	Tomas Chaparro	5
#22 – Research Linux utilization & storage	In Progress	Taha Anwar & Tomas Chaparro	5

#43 – Add multi-file upload	In Progress	Taha Anwar	5
#59 – Dockerize Project	Complete	John-Paul Osele	8
#57 – File Filtering and Sorting	Complete	Tomas Chaparro	5

Estimated Velocity (At beginning of Sprint)	48
Calculated Velocity (At end of Sprint)	24

Week 3 Progress

Scrum Master: Tomas Chaparro

Sprint Goal: Finalize the functionality of The Vault Keeper, prepare a final release, and prepare/deliver a final presentation.

Backlog Features

- User Management: Add user accounts and complete security logic.
- UI/UX Fixes: Functionality to make the user experience more visually clear.
- Presentation Creation: Create the final presentation for the final demo.

Tasks in Sprint	Task Status at end of Sprint	Assigned To	Size
#45 – Add confirmation modals – V2	Completed	Cole Roberts	3
#43 – Add multi-file upload	Completed	Tomas Chaparro	5
#54 – Finalize UI/UX Design – V8	Completed	Cole Roberts	8
#55 – Complete Scrum Review Documentation	Completed	Taha Anwar	3
#57 – File filtering and sorting	Completed	Tomas Chaparro	5
#59 – Dockerize Project	Completed	John-Paul Osele	5
#42 – Make mobile friendly – V2	Completed	Cole Roberts	5
#30 – Password Protected Files	Completed	Tomas Chaparro	5
#63 – Create the Final Presentation	Completed	Taha Anwar	8

#41 – Change website title and favicon – V2	In-Progress	Cole Roberts	1
#40 – Add user accounts	Not Started	Team	5
#44 – Fix google translate bug	Not Started	Team	3
#46 – Fix visual bug	Not Started	Team	3
#47 – Add mini preview for files	Not Started	Team	3
#29 – Drag and Drop	Not Started	Team	5
#31 – Add authentication	Not Started	Team	5

Estimated Velocity (At beginning of Sprint)	68
Calculated Velocity (At end of Sprint)	45

Week 4 Progress

Scrum Master: Taha Anwar

Sprint Goal: Complete automated testing, documentation, and repo additions.

Backlog Features

- Automated Testing: Create testing for The Vault Keeper to check functionality.
- Documentation: Finalize the documentation, including UMLs, Scrum Boards, and weekly sprint progression.
- Update Readme: Update the current Read.me on the front page of the repository with correct instructions to run.

Tasks in Sprint	Task Status at end of Sprint	Assigned To	Size
#64 – Complete automated testing	Completed	Cole Roberts	3
#65 – Finalize Documentation	Completed	Taha Anwar	5
#66 – Clean the Repository & Update Readme	Completed	John-Paul Osele	1
#67 – Create the Official Release	Completed	Tomas Chaparro	1

Estimated Velocity (At beginning of Sprint)	10
Calculated Velocity (At end of Sprint)	10